



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche,
Informatiche e Matematiche

8. LAB 3: Loops e UD/DU Chains

Linguaggi e Compilatori [I215-011]

Corso di Laurea in INFORMATICA
(D.M.270/04) [16-262]
Anno accademico 2023/2024

Prof. Andrea Marongiu
andrea.marongiu@unimore.it

Copyright note

È vietata la copia e la riproduzione dei contenuti e immagini in qualsiasi forma.

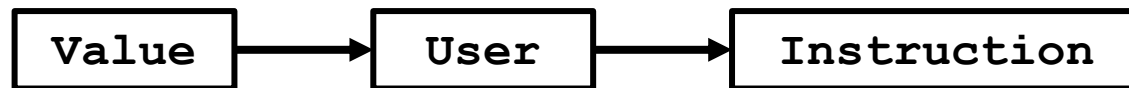
È inoltre vietata la redistribuzione e la pubblicazione dei contenuti e immagini non autorizzata espressamente dall'autore o dall'Università di Modena e Reggio Emilia.

Credits

- Gibbons, Carnegie Mellon University, “Optimizing Compilers”
- Pekhimenko, University of Toronto, “Compiler Optimization”

UD e DU Chains

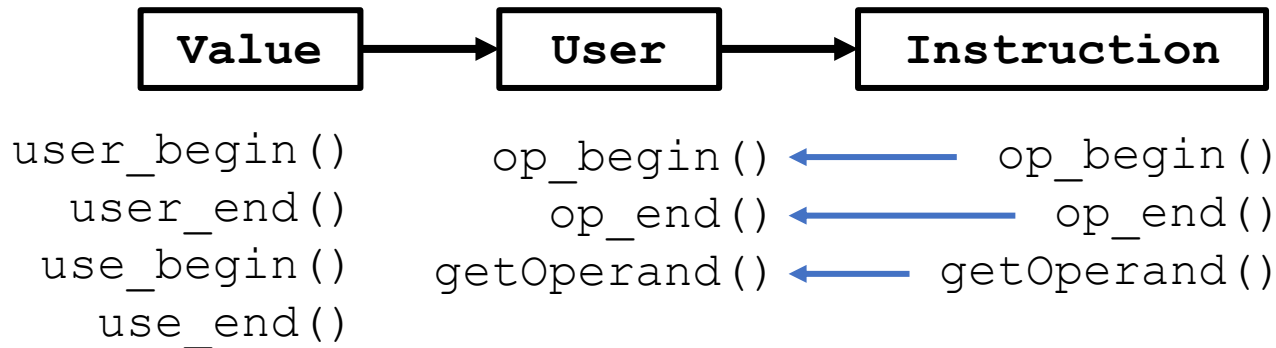
- Ricordate la relazione *User – Use – Value* in LLVM?
- Le istruzioni (`Instruction`) LLVM ereditano dalla classe `Value`
- Ma ereditano anche dalla classe `User`
 - Quindi esiste implicitamente un legame tra le **istruzioni** e i loro **usi**
- In altre parole, le `Instruction` giocano entrambi i ruoli di **User** e **Usee** (`Value`)



GERARCHIA DELLE CLASSI LLVM

UD e DU Chains

GERARCHIA DELLE CLASSI LLVM



- Data una definizione (`Value`) è facile scorrere i suoi usi con gli iteratori della classe base (UD Chain)
- Dato un uso (operando), è facile derivare la sua *reaching definition* (DU Chain): in LLVM (SSA) corrisponde all'uso stesso
 - Ricorda: un operando (`Value`) può essere castato al tipo `Instruction`

Ricorda: Proprietà della forma SSA

- Solo un assegnamento per variabile
- Le definizioni dominano gli usi
- Un cambio di filosofia sull'analisi e ottimizzazione
- Se rimuoviamo la possibilità di riassegnare valori alle variabili alcuni concetti si fondono:
 - Definizioni = Variabili
 - Istruzioni = Valori
 - Operandi = Archi di un DFG



Da qui la rappresentazione di LLVM

Ricorda: Istruzioni come Usee

Perché un oggetto `Instruction` è anche uno **Usee**?

- La risposta risiede nel come dobbiamo interpretare un oggetto `Instruction` LLVM:

```
%2 = add %1, 0
```

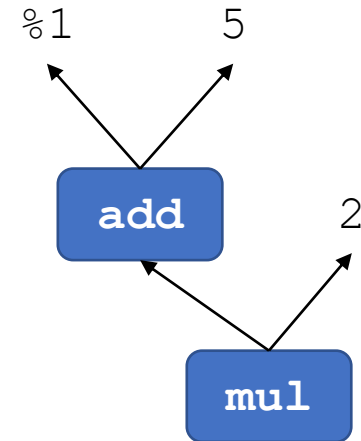
-  Il risultato dell'istruzione `add %, 0` viene assegnato a `%2`
-  `%2` è la rappresentazione `Value` dell'istruzione `add %1, 0`

- Quindi, ogni volta che nel testo usiamo il valore `%2` in realtà intendiamo proprio indicare l'istruzione `add %1, 0`

Ricorda: Proprietà della forma SSA

**In LLVM è proprio un puntatore
allo stesso oggetto**

```
%3 = add nsw i32 %1, 5  
%4 = mul nsw i32 %3, 2
```



- Definizioni = Variabili
- Istruzioni = Valori
- Operandi = Archi di un DFG

**Da qui la rappresentazione
di LLVM**

User – Use – Value in LLVM

```
Instruction &Inst = ...;
```

```
outs() << "I am the instruction: " << Inst << "\n";  
// print the instruction as an operand  
outs() << "Me as an operand: ";  
Inst1st.printAsOperand(outs(), false);  
outs() << "\n";
```



```
%3 = add nsw i32 %1, 1  
Me as an operand: %3
```



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche,
Informatiche e Matematiche

LAB3 – LoopWalk Pass

loop in LLVM

- Come abbiamo visto precedentemente, è possibile scrivere passi LLVM che operano direttamente sui loop (`LoopPass`)
- L'entry point in un passo simile è la funzione

```
PreservedAnalyses <PassName>::run(Loop &L,  
                                     LoopAnalysisManager &LAM,  
                                     LoopStandardAnalysisResults &LAR,  
                                     LPMUpdater &LU)
```

che possiede un handle all'oggetto `Loop &L`
- La classe `Loop` eredita dalla classe `LoopBase`, che contiene diversi metodi utili per analizzare il loop



GERARCHIA DELLE CLASSI LLVM

I loop in LLVM

- Verificare che il loop sia in forma normale

```
isLoopSimplifyForm ()
```

- Recuperare blocchi significativi del loop

```
getLoopPreheader () const
```

```
getHeader () const
```

```
getBlocks () const
```

```
...
```

- Scorrere i basic blocks che compongono un loop

```
for (Loop::block_iterator BI = L->block_begin(); \
      BI != L->block_end(); \
      ++BI)
```